EXP: 5 CLASS DIAGRAM

AIM: To design and develop a Class Diagram for the system "Precision Disaster Recovery (PDR-Sync)" representing system structure, attributes, methods, and relationships using UML.

THEORY:

A Class Diagram is a type of UML diagram used to represent the static structure of a system. It shows:

- Classes
- Attributes
- Methods (functions)
- Relationships between classes

Components of Class Diagram

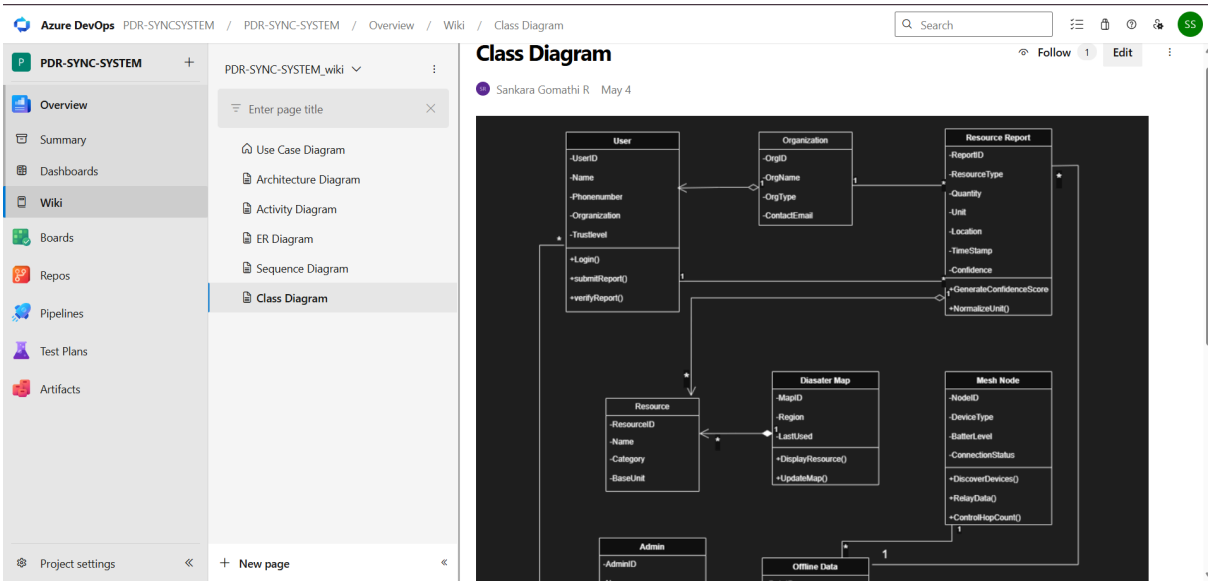
- Class: Represents an entity in the system
  (Example: User, Resource, Report)
- Attributes: Data members of a class
  (Example: Name, ID, Timestamp)
- Methods: Functions or operations of a class
  (Example: login(), syncData())
- Relationships
  - Association → General relationship
  - Aggregation → Weak relationship (has-a)
  - Composition → Strong relationship
  - Multiplicity → One-to-many (1..*), etc.

NOTATIONS:

Notation	Description
Rectangle (3 parts)	Class (Name, Attributes, Methods)
Line	Association
Arrow	Direction
Diamond	Aggregation / Composition
1, *	Multiplicity

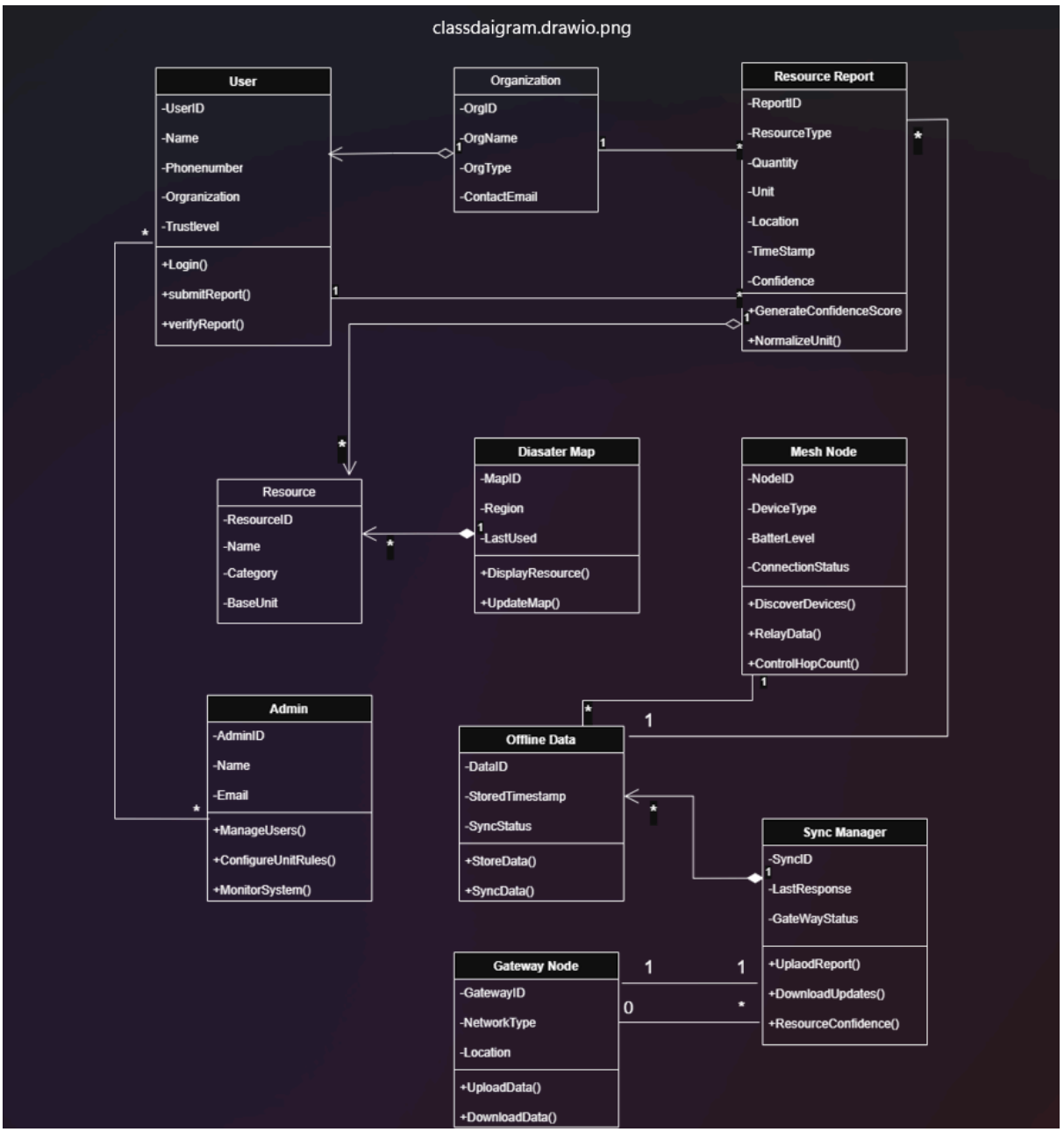
PROCEDURE:

1. Identify all major entities (classes) in the system
   (User, Resource, Report, Admin, Sync Manager, etc.)
2. Define attributes for each class
3. Define methods/functions for each class
4. Establish relationships between classes
5. Add multiplicity (1, many)
6. Draw the diagram using UML standards



Steps to Create Class Diagram in Azure

1. Login to Azure Portal
2. Open your Azure DevOps Project
3. Navigate to:
   - Repos / Wiki
4. Use diagram tools:
   - draw.io (diagrams.net)
   - Lucidchart
5. Create classes with:
   - Attributes
   - Methods
6. Connect classes using relationships
7. Export diagram as image
8. Upload to Azure DevOps



Explanation of Class Diagram

The system consists of multiple interacting classes:

- User
  - Attributes: UserID, Name, PhoneNumber, TrustLevel
  - Methods: login(), submitReport(), verifyReport()
- Organization
  - Stores organization details
  - Linked with multiple users

- Resource Report
  - Stores disaster resource data
  - Methods:
    - generateConfidenceScore()
    - normalizeUnit()
- Resource
  - Defines resource types and base units
- Disaster Map
  - Displays resources and updates map
- Mesh Node
  - Handles device communication
  - Methods:
    - discoverDevices()
    - relayData()
    - controlHopCount()
- Offline Data
  - Stores data locally
  - Syncs when network available
- Sync Manager
  - Manages upload/download
  - Handles synchronization
- Gateway Node
  - Acts as connection point to server
- Admin
  - Manages users and system rulesA

RESULT:

Thus, the Class Diagram for the system *"Precision Disaster Recovery (PDR-Sync)"* was successfully designed, representing all classes, attributes, methods, and relationships.